

Contents

Audio Chord Recognition Using Convolutional Neural Networks	2
Abstract	2
1. Introduction	2
2. Music Theory Background	2
2.1 What is a Chord?	2
2.2 Root Notes	3
2.3 Chord Quality	3
2.4 Why Root-Only Classification First?	3
3. Data Pipeline and Preprocessing	3
Pipeline Overview	3
3.1 Data Collection	4
3.2 Data Cleaning	4
3.3 Data Augmentation	5
3.4 Label Normalization	5
4. Feature Extraction Evolution	5
4.1 Why Spectral Features?	5
4.2 Feature Extraction Methods	5
4.3 Feature Comparison Summary	9
5. CNN Architecture Design	9
5.1 Design Philosophy: Music is Not a Natural Image	9
5.2 Architecture Overview	10
5.3 Key Architectural Features	11
5.4 Training Configuration	11
6. Codebase Evolution	12
6.1 Phase 1: Cluster-Based Approach (<code>Chord_Recognition_Old/</code>)	12
6.2 Phase 2: Kaggle Refactoring (<code>Chord_Recognition/</code>)	13
6.3 Comparison: Old vs New	14
6.4 Lessons Learned	14
7. Results and Analysis	14
7.1 Performance Summary	14
7.2 Feature Type Analysis	15
7.3 Training Dynamics Analysis	16
7.4 Task Difficulty Comparison	17
7.5 Computational Efficiency	17
8. Discussion and Future Work	17
8.1 What Worked Well	17
8.2 Limitations and Challenges	17
8.3 Future Improvements	18
8.4 Broader Implications	19
9. Conclusion	19
10. References	20
Libraries and Tools	20
Dataset	20
Code Repository Structure	20
Hyperparameter Configuration	21

Audio Chord Recognition Using Convolutional Neural Networks

Deep Learning Final Project Report

Killian Bertsch 12/5/2025

Abstract

This project implements a convolutional neural network (CNN) system for recognizing jazz chords from audio recordings. The system processes 1-bar audio clips through spectral feature extraction and performs two classification tasks: (1) root note classification into 12 classes (C, Db, D, Eb, E, F, Gb, G, Ab, A, Bb, B), and (2) combined root+quality classification into 40 classes. Four different feature extraction methods were evaluated: Chromagram (12 bins), Hybrid (36 bins), CQT-60 (60 bins), and CQT-84 (84 bins). The best-performing models achieved 79.61% validation accuracy on root-only classification using Hybrid features, and 67.12% validation accuracy on root+quality classification using Chroma features, significantly outperforming random baselines of 8.3% and 2.5% respectively. The project demonstrates the importance of domain-specific feature engineering and CNN architecture design for audio classification tasks.

1. Introduction

This project addresses the challenge of automatic musical chord recognition from audio—a fundamental problem in Music Information Retrieval (MIR). The system is designed to classify 1-bar audio clips into chord labels, focusing on two distinct tasks: identifying the root note alone (12-class problem), and identifying both root note and chord quality (40-class problem).

The approach leverages convolutional neural networks applied to spectral representations of audio, treating chord recognition as a specialized form of image classification. Four feature extraction methods were systematically compared, revealing that domain-specific feature engineering based on music theory principles substantially outperforms generic approaches. The project also demonstrates practical machine learning engineering through a complete pipeline from raw audio collection through feature extraction, model training, and evaluation on a cloud platform.

2. Music Theory Background

For readers unfamiliar with music theory, this section provides essential background for understanding the classification task.

2.1 What is a Chord?

A chord is three or more notes played simultaneously. Chords form the harmonic foundation of music. Each chord has two primary components:

1. **Root Note** - The “home base” or fundamental note that gives the chord its name
2. **Chord Quality** - The “flavor” or “color” of the chord, determined by the intervals between notes

2.2 Root Notes

There are 12 possible root notes in Western music, corresponding to the 12 pitch classes in an octave:

C, Db, D, Eb, E, F, Gb, G, Ab, A, Bb, B

This forms the basis of our 12-class classification problem. In Western music theory, the root note is the lowest frequency, placed in the bass register. It is the first determining factor of harmony, establishing the tonal center around which other chord tones are understood.

2.3 Chord Quality

Chord quality describes the intervallic structure of a chord. Common qualities include:

- **Major (maj):** Intervals of major third and perfect fifth (example: C-E-G)
- **Minor (min):** Intervals of minor third and perfect fifth (example: C-Eb-G)
- **Dominant (dom):** Major triad with minor seventh (example: C-E-G-Bb)
- **Half-diminished (hdim):** Diminished triad with minor seventh (example: C-Eb-Gb-Bb)
- **Diminished (dim):** Fully diminished chord (example: C-Eb-Gb-A)

For this project, chords with extensions (9ths, 11ths, 13ths) were normalized to their base quality. For example, Cmaj7, Cmaj9, and Cmaj13 all map to “C:maj”. This reduced the root+quality classification to 40 classes (12 roots $\times$ ~3-4 common qualities).

2.4 Why Root-Only Classification First?

The project prioritizes root-only classification for several reasons:

1. **Music-Theoretic Importance:** The root defines the chord’s identity and function in harmonic progressions
2. **Simpler Problem:** 12 classes vs 40+ classes for full chord classification
3. **Strong Foundation:** Success at root recognition provides a basis for more complex classification

3. Data Pipeline and Preprocessing

The data pipeline transforms raw audio recordings into structured datasets suitable for CNN training. The process consists of data collection, cleaning, augmentation, and feature extraction.

Pipeline Overview

Raw Audio (00_raw/)

↓

[Load & Parse] → Read chord progressions, load audio slices

↓

[Clean & Organize] → Deduplicate slices, split bars, create sample IDs

↓

[Augmentation 4x] → Pitch shift: -2, 0, +2, +4 semitones, transpose labels

↓

[Feature Extraction] → Extract Chroma/Hybrid/CQT60/CQT84 ($n_{\text{bins}} \times 87$ frames)
↓
[CSV Export] → Flatten features, add metadata → data_output/*.csv
↓
[Kaggle Training] → Load CSV, grouped split, train CNN, evaluate

3.1 Data Collection

The dataset was created by pairing audio recordings with chord progressions from iReal Pro charts. Eleven jazz standards were used:

- All Of You (32 bars)
- Autumn Leaves (32 bars)
- Beatrice (16 bars)
- Bemsha Swing (16 bars)
- Black and Blue (32 bars)
- Blue Bossa (16 bars)
- Blues - Minor (12 bars)
- Days of Wine and Roses (32 bars)
- For Sentimental Reasons (32 bars)
- Softly As In A Morning Sunrise (32 bars)
- Stella By Starlight (32 bars)

Total: 284 bars → ~550 individual chord samples (before augmentation)

Audio was exported from Ableton Live (an audio editing program) as bar-by-bar slices, creating time-aligned pairs of audio files and chord labels.

3.2 Data Cleaning

The first preprocessing step handles export artifacts:

3.2.1 Deduplication

During the export process, duplicate slices were accidentally created when looping. The script uses timestamps embedded in filenames to identify and remove duplicates.

3.2.2 Bar Splitting

Jazz chord progressions often have two chords per bar (e.g., “Dm7 | Cm7”). The system automatically splits these into two separate files: - song_bar27a.wav (first chord) - song_bar27b.wav (second chord)

3.2.3 Cycle Handling

In jazz music, the musicians will generally improvise over a looped form. This means that for any audio recording, there are multiples clips corresponding with each bar of the form. For example, a form may only be 16 bars, so the 17th bar of the recording will correspond with Bar 1 again.

When a song form repeats, later occurrences receive cycle numbers: - song_bar01.wav (first occurrence) - song_bar01_cycle02.wav (second occurrence)

All labels follow the format root:quality (e.g., F:maj, G:min).

3.3 Data Augmentation

To expand the limited dataset, pitch-shift augmentation was applied with shifts of **-2, 0, +2, +4 semitones**, resulting in a 4x increase in dataset size.

This approach leverages a fundamental principle of music theory: the same chord in different keys has identical harmonic relationships, just at different absolute frequencies. For example:

- Original: `beatrice_bar01.wav` → Label: `F:maj`
- Shifted +2: `beatrice_bar01_shift+2.wav` → Label: `G:maj` (transposed)

The augmentation provides genuine variation while maintaining musical validity. Critically, **grouped train/validation splitting** ensures that all augmented versions of a bar stay in the same split, preventing data leakage that would artificially inflate validation accuracy.

Final Dataset: - Original samples: ~550 - After augmentation (4x): ~2,200 samples - Train/validation split: 80/20 using grouped splitting

3.4 Label Normalization

Chord labels were parsed and normalized to a consistent format:

- Sharp symbols converted to flats: `F#` → `Gb`, `C#` → `Db`
- Extended chord qualities mapped to base forms: `Cmaj9` → `C:maj`
- Two-character quality identifiers: `maj`, `min`, `dom`, `hdim`, `dim`

This normalization reduced the complexity of the root+quality classification from 100+ possible combinations to 40 classes.

4. Feature Extraction Evolution

A central challenge in audio machine learning is converting raw audio waveforms into representations suitable for neural networks. This project explored four different feature extraction methods, each informed by music theory and signal processing principles.

4.1 Why Spectral Features?

Raw audio waveforms are time-domain signals that don't directly reveal frequency content. However, chords are defined by **simultaneous frequencies**. Spectral features convert audio into a time-frequency representation—essentially creating a 2D image where:

- **Vertical axis** = Frequency (which notes are playing)
- **Horizontal axis** = Time (when notes change)
- **Intensity** = Energy at each frequency

This transformation enables treating audio classification as a computer vision problem, allowing CNNs to “see” musical patterns.

4.2 Feature Extraction Methods

All features were extracted using the `librosa` library and normalized to a consistent shape of (`n_bins`, 87 frames), where `n_bins` varies by feature type and 87 frames represents approximately

one bar of music at typical tempos.

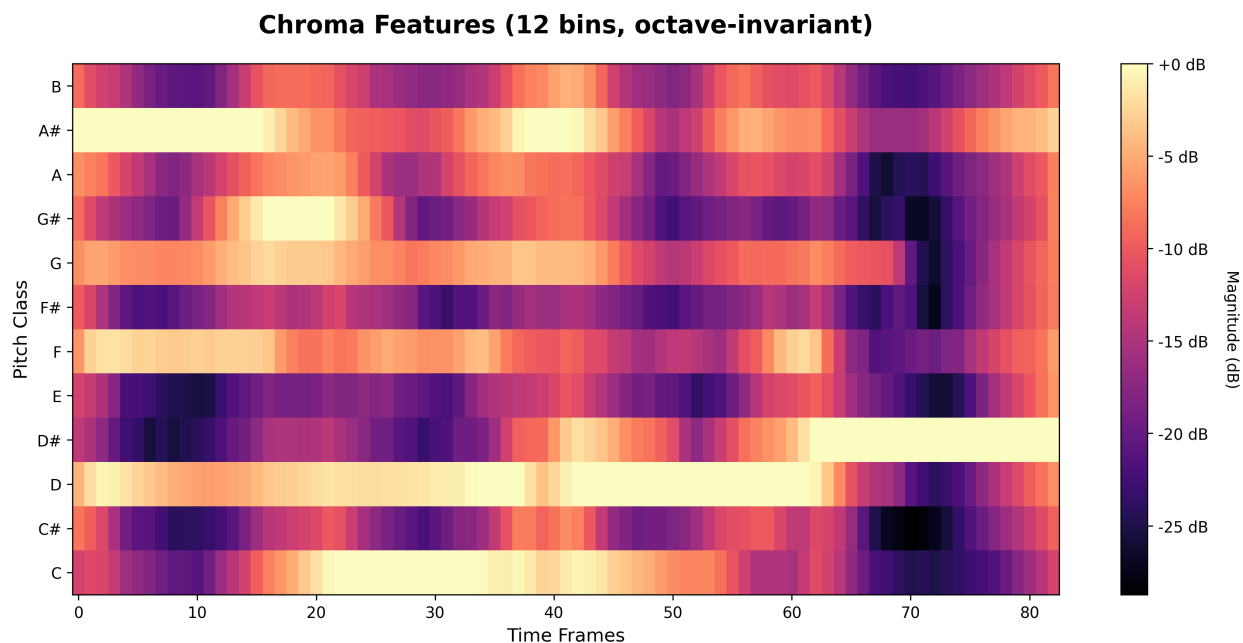


Figure 1: Chroma Feature Visualization

4.2.1 Chromagram (Chroma) - 12 bins **Description:** Chromagram is an octave-invariant representation that collapses all frequencies into 12 pitch classes (C, C#, D, ..., B).

How it works: All instances of “C” (C2, C3, C4, etc.) contribute to the same “C” bin, regardless of octave.

Advantages: - Smallest feature size (12 bins) → fastest training - Directly maps to pitch classes → intuitive for chord recognition - Focuses on harmonic content without octave information

Problem Identified: Root notes are typically placed in the **bass register** (low octaves). Chromagram collapses bass notes together with higher octave notes, making it difficult to distinguish the root from other chord tones. The root note’s position in the bass is partially lost in the octave collapse.

Performance: - Root-only: 68.10% validation accuracy - Root+quality: 67.12% validation accuracy (best for quality!)

4.2.2 CQT-84 (Full Constant-Q Transform) - 84 bins **Description:** Constant-Q Transform with 84 frequency bins spanning 7 octaves (C1 to B7).

How it works: CQT is a logarithmic frequency transform where each bin represents a musical note. Unlike standard Fourier transforms (which use linear frequency scales), CQT’s logarithmic scale matches how humans perceive pitch.

Advantages: - Preserves all frequency information across the audible range - Bass notes are separate from treble → root should be distinguishable - Music-theoretically aligned representation

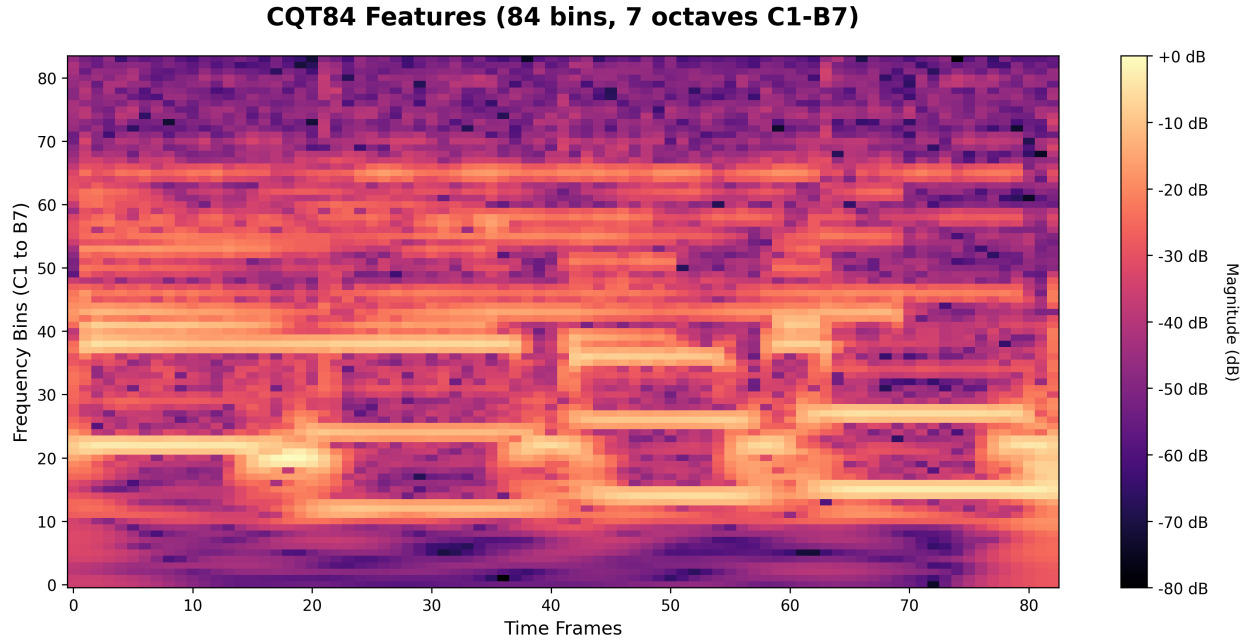


Figure 2: CQT84 Feature Visualization

Problem Identified: - High frequencies (above C6) contribute little to chord identification - Larger feature size (84 bins) → slower training → more overfitting risk - High-frequency noise from harmonics and room acoustics

Performance: - Root-only: 55.78% validation accuracy (worst!) - Root+quality: 49.50% validation accuracy - Highest overfitting gap: 38.13%

The poor performance suggests that more information doesn't always help, especially with limited training data.

4.2.3 CQT-60 (Reduced Constant-Q Transform) - 60 bins **Description:** Constant-Q Transform with 60 frequency bins spanning 5 octaves, removing the highest frequencies.

Rationale: Since CQT-84's high frequencies appeared to hurt performance, CQT-60 removes these potentially noisy bins while retaining the musically relevant frequency range.

Advantages: - Faster training than CQT-84 - Less high-frequency noise - Still preserves bass/treble distinction

Performance: - Root-only: 69.48% validation accuracy (better than CQT-84 and Chroma) - Root+quality: 53.47% validation accuracy - Moderate training time: 9.7 minutes

This validated the hypothesis that high frequencies were more harmful than helpful.

4.2.4 Hybrid (Low-CQT + Chroma) - 36 bins [WINNER] **Description:** A custom feature that vertically stacks: - 24-bin Low-CQT (2 octaves in the bass region) - 12-bin Chromagram

Rationale: This feature was designed based on the following musical insight: - **Root notes** are in the bass → preserved by Low-CQT - **Chord quality** is about interval relationships → captured by

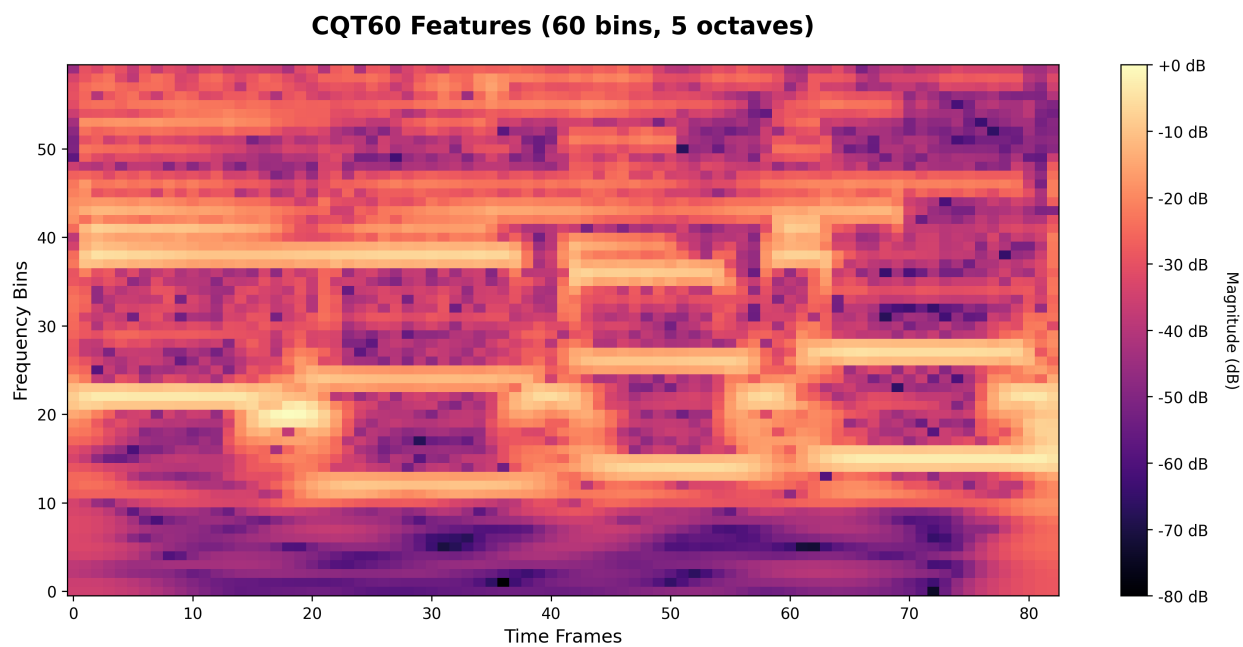


Figure 3: CQT60 Feature Visualization

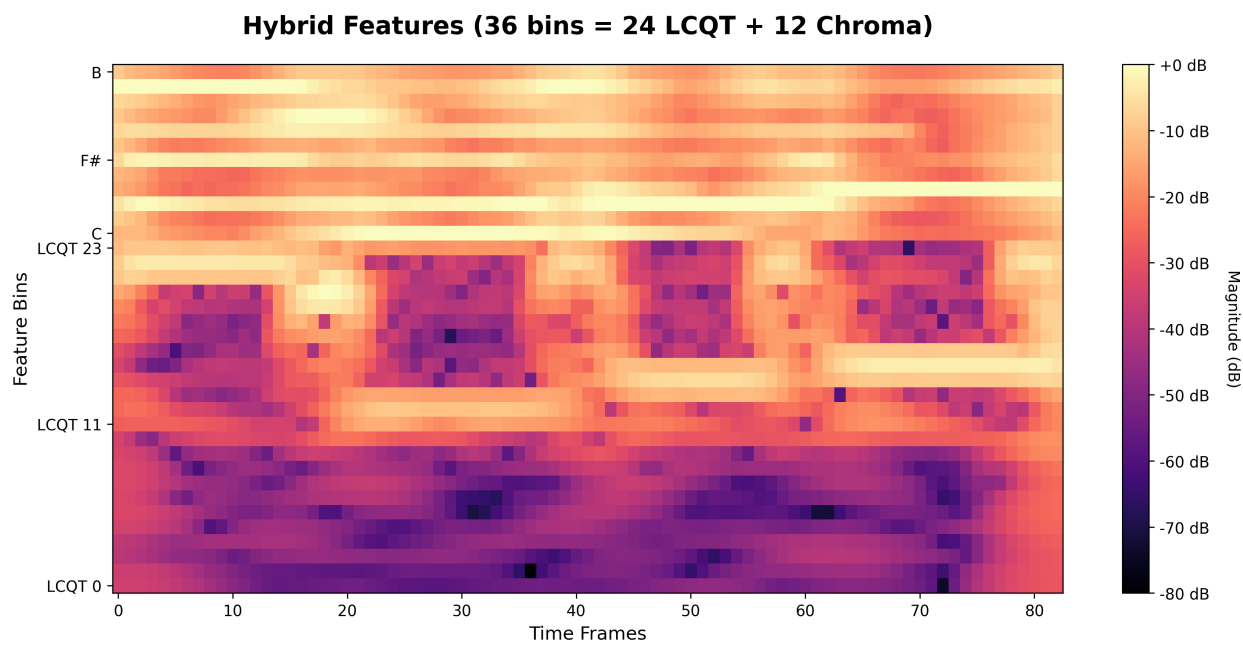


Figure 4: Hybrid Feature Visualization

Chroma's octave-invariant representation - Combining both provides complementary information

Implementation:

```
# Extract low-CQT (24 bins = 2 octaves starting from C1)
lcqt = librosa.cqt(y, n_bins=24, bins_per_octave=12, fmin=librosa.note_to_hz('C1'))

# Extract chromagram (12 bins)
chroma = librosa.feature.chroma_cqt(y, n_chroma=12)

# Stack vertically
hybrid = np.vstack([lcqt, chroma]) # Shape: (36, time)
```

Advantages: - Best of both worlds: bass detail + harmonic summary - Smaller than CQT-60/84 but more informative than pure Chroma - Musically motivated design

Performance: - Root-only: **79.61% validation accuracy** (best!) - Root+quality: 57.21% validation accuracy - Training time: 6.0 minutes (balanced) - Lowest overfitting gap for root classification: 20.37%

The Hybrid approach's success demonstrates the value of domain-specific feature engineering.

4.3 Feature Comparison Summary

Feature	Bins	Root Val Acc	Quality Val Acc	Training Time	Key Insight
Hybrid	36	79.61%	57.21%	6.0 min	Bass + harmony = best roots
CQT-60	60	69.48%	53.47%	9.7 min	Less is more (vs CQT-84)
Chroma	12	68.10%	67.12%	2.3 min	Best for quality detection
CQT-84	84	55.78%	49.50%	7.8 min	Too much info → overfitting

5. CNN Architecture Design

The CNN architecture is specifically designed to exploit the structure of musical spectrograms, where chords appear as vertical patterns (simultaneous frequencies) evolving horizontally over time.

5.1 Design Philosophy: Music is Not a Natural Image

A critical insight for this project is that musical spectrograms have fundamentally different structure than natural images:

In Natural Images: - Patterns can appear anywhere in 2D space - Vertical and horizontal directions are equally important - Square kernels (3×3 , 5×5) work well

In Musical Spectrograms: - **Vertical axis** = Frequency (which notes are playing simultaneously)
- **Horizontal axis** = Time (when notes change) - **Chords** = VERTICAL patterns (multiple frequencies at the same instant) - **Chord progressions** = HORIZONTAL patterns (chords changing over time)

This structural difference motivated custom architectural choices.

5.2 Architecture Overview

```
Input: (batch, 1, n_bins, 87)
↓
[Conv1] 5×1 kernel, 32 filters
      → BatchNorm → ReLU → MaxPool 2×2
↓
[Conv2] 3×3 kernel, 64 filters
      → BatchNorm → ReLU → MaxPool 2×2
↓
[Conv3] 3×3 kernel, 128 filters
      → BatchNorm → ReLU → MaxPool 2×2
↓
[Conv4] 3×3 kernel, 256 filters
      → BatchNorm → ReLU
↓
[AdaptiveAvgPool] → (batch, 256, 1, 1)
↓
Flatten → (batch, 256)
↓
[FC1] 256 → 256 + ReLU + Dropout(0.5)
↓
[FC2] 256 → 128 + ReLU + Dropout(0.5)
↓
[FC3] 128 → num_classes
↓
Output: (batch, num_classes)
      → 12 classes (root-only) or 40 classes (root+quality)
```

The CNN architecture consists of:

1. **Convolutional Feature Extraction:** 4 convolutional layers with increasing filter counts (32→64→128→256)
2. **Adaptive Pooling:** Handles variable input sizes from different feature types
3. **Fully Connected Classification:** 3 dense layers with dropout for regularization

Input Shape: (batch_size, 1, n_bins, 87) - 1 channel (grayscale spectrogram) - n_bins varies: 12 (Chroma), 36 (Hybrid), 60 (CQT-60), 84 (CQT-84) - 87 time frames

Output Shape: (batch_size, num_classes) - num_classes = 12 for root-only - num_classes = 40 for root+quality

5.3 Key Architectural Features

5.3.1 Vertical Kernel in First Layer The first convolutional layer uses a **5×1 kernel** instead of the standard 3×3 or 5×5:

```
self.conv1 = nn.Conv2d(1, 32, kernel_size=(5, 1), padding=(2, 0))
```

Rationale: - A chord is defined by **simultaneous frequencies** (vertical pattern in spectrogram) - A 5×1 kernel looks at 5 adjacent frequency bins at one instant in time - This captures harmonic intervals (the distance between notes in a chord) - Avoids blurring across time, which would mix different chords

Musical Interpretation: The 5×1 kernel can detect intervals like: - Major third (C to E = 4 semitones) - Perfect fifth (C to G = 7 semitones) - Seventh (C to B = 11 semitones)

These intervals define chord quality, so detecting them early in the network is musically meaningful.

5.3.2 Asymmetric Pooling The first pooling layer uses **2×2 pooling** instead of 2×1 or uniform pooling:

```
self.pool1 = nn.MaxPool2d(kernel_size=(2, 2))
```

Rationale: - Reduces frequency resolution (less critical—harmonics are redundant) - Preserves some time resolution (important because jazz chords change quickly) - Provides computational efficiency without losing musically relevant information

5.3.3 Adaptive Pooling for Multi-Feature Support To support different feature types (12, 36, 60, 84 bins) with the same architecture:

```
self.adaptive_pool = nn.AdaptiveAvgPool2d((1, 1))
```

Advantage: - Collapses any spatial size to 1×1 - Allows the same model to handle all feature types - No architecture redesign needed when changing features

5.3.4 Dropout Regularization With a relatively small dataset (~2,200 samples), overfitting is a concern. Dropout layers (50%) are applied after each fully connected layer:

```
self.fc1 = nn.Linear(256, 256)
self.dropout1 = nn.Dropout(0.5)
```

This helps prevent the model from memorizing the training set.

Parameter Counts: - Chroma (12 bins): ~107,000 parameters - Hybrid (36 bins): ~180,000 parameters - CQT-60 (60 bins): ~220,000 parameters - CQT-84 (84 bins): ~250,000 parameters

5.4 Training Configuration

Loss Function: CrossEntropyLoss (for multi-class classification)

Optimizer: Adam with initial learning rate of 0.001

Learning Rate Scheduling: ReduceLROnPlateau - Reduces LR by 50% if validation loss doesn't improve for 5 epochs - Helps fine-tune in later epochs

Early Stopping: Patience of 10 epochs - Stops training if validation accuracy doesn't improve for 10 epochs - Prevents unnecessary computation and overfitting

Batch Size: 32 (balanced for GPU memory and gradient stability)

Epochs: Up to 100, but typically early-stopped around 20-40 epochs

6. Codebase Evolution

The project underwent a significant refactoring process, evolving from a complex cluster-based workflow to a streamlined Kaggle-friendly system. Understanding this evolution provides insight into practical machine learning engineering.

6.1 Phase 1: Cluster-Based Approach (Chord_Recognition_Old/)

Initial Context: The project began with the intention to train models on the university's Hercules computing cluster, which offered more computational resources than a laptop.

Architecture: - **7 pipeline scripts** numbered 00-07 for sequential execution - **5 data directories** representing pipeline stages: - `00_raw/` - Original Ableton exports - `01_interim/` - Cleaned and deduplicated data - `02_processed/` - Augmented data with pitch shifts - `03_features/` - Extracted features as `.npy` files - `experiments/` - Trained models and results

Directory Structure:

```
Chord_Recognition_Old/
  pipelines/
    01_clean_data.py
    02_augment_data.py
    03_extract_features.py
    04_train_model.py
    06_compare_models.py
    07_predict.py
  src/
    data/ (chord_utils, audio_utils, file_utils)
    features/ (extractors)
    models/ (chord_cnn)
    training/ (dataset)
  data/ (5-stage pipeline)
  experiments/models/
```

Workflow:

```
# Run each pipeline script sequentially
python pipelines/01_clean_data.py
python pipelines/02_augment_data.py
python pipelines/03_extract_features.py --feature_type hybrid
python pipelines/04_train_model.py --feature_type hybrid
python pipelines/06_compare_models.py
```

Advantages: - Clear separation of concerns (each script does one thing) - Modular design with reusable `src/` library code - Can skip already-completed stages - Easy to debug individual components

Challenges: - Everything was messy and took a long time to set up - Complex dependencies between 7 different scripts - `.npz` files not portable or cloud-friendly - Difficult to share with collaborators - Cluster access required SSH, file transfers, job scheduling - Hard to version control large `.npz` files

6.2 Phase 2: Kaggle Refactoring (`Chord_Recognition/`)

Motivation: After the initial implementation worked, I remembered the 30 hours per week of P100 GPU time. This was more accessible than the university cluster, but required a different data format.

Key Insight: Kaggle works best with CSV files, not individual `.npz` arrays.

Refactoring with Claude AI: I used Claude AI to help refactor the codebase for Kaggle deployment. The refactoring involved:

1. **Simplification:**
 - 7 pipeline scripts → 1 Jupyter notebook (`data_preprocessing.ipynb`)
 - 5 data directories → 2 directories (`data_input/`, `data_output/`)
2. **Format Conversion:**
 - Individual `.npz` files → CSV files with flattened features
 - Each row in CSV = one audio sample with metadata + flattened feature array
3. **Workflow Streamlining:**
 - All preprocessing happens in one notebook
 - Upload CSV to Kaggle Datasets
 - Train on Kaggle GPU
 - Download results

New Directory Structure:

```
Chord_Recognition/
  data_preprocessing.ipynb    # All preprocessing in ONE notebook
  model_training.ipynb       # Training (runs on Kaggle)
  data_input/
    00_raw/                  # Copy raw data here
  data_output/               # CSV outputs
    chroma_features.csv
    hybrid_features.csv
    cqt60_features.csv
    cqt84_features.csv
```

CSV Structure: Each CSV file contains flattened features with metadata:

id	label	root	quality	song	shift	feature_0	feature_1	...	feature_N
beatrice_01	F	maj		beatrice 0		-12.5	-15.3	...	-20.1
beatrice_01	G	maj	Shift+2	beatrice 2		-13.1	-14.8	...	-18.7

Benefits: - **Much easier to manage:** One notebook vs 7 scripts - **Cloud-ready:** CSV files

upload directly to Kaggle - **Portable:** Share datasets easily - **No cluster access needed:** Free Kaggle GPU - **Reproducible:** Everything in one place - **Version control friendly:** CSV files can be tracked or excluded cleanly

What Was Preserved: All the core logic remained identical: - Same preprocessing algorithms - Same feature extraction code - Same augmentation strategy - Same grouped train/val splitting - Same CNN architecture - Same training procedure

Only the **data organization** changed, not the underlying machine learning.

6.3 Comparison: Old vs New

Aspect	Old (Cluster)	New (Kaggle)
Scripts	7 pipeline files	1 notebook
Data folders	5 stages	2 folders
File format	.npy (binary)	.csv (text)
Training location	University cluster	Kaggle cloud
Complexity	High	Low
Portability	Difficult	Easy
Sharing	Hard	Simple (upload CSV)
Iteration speed	Slow (cluster jobs)	Fast (interactive notebook)

6.4 Lessons Learned

1. **Start simple, add complexity only when needed** - The initial complex architecture was over-engineered for a 1,880-sample dataset
2. **Data format matters** - CSV's simplicity and ubiquity make it ideal for cloud platforms
3. **Platform constraints drive design** - Kaggle's CSV-friendly ecosystem influenced the refactoring
4. **AI-assisted refactoring** - Claude AI helped restructure code while preserving functionality
5. **Core algorithms > workflow complexity** - The same ML performed well in both systems; workflow is just packaging

7. Results and Analysis

7.1 Performance Summary

All models were trained on Kaggle's P100 GPU using the refactored CSV-based workflow. Training used early stopping with patience of 10 epochs and learning rate reduction when validation loss plateaued.

Root-Only Classification Results (12 classes):

Feature Type	Best Val Acc (%)	Final Val Acc (%)	Final Train Acc (%)	Overfitting Gap (%)	Epochs	Time (min)
Hybrid	79.61	77.84	98.21	20.37	33	6.0
CQT60	69.48	68.70	96.48	27.78	35	9.7
Chroma	68.10	66.50	95.78	29.28	26	2.3
CQT84	55.78	50.89	89.02	38.13	21	7.8

Root+Quality Classification Results (40 classes):

Feature Type	Best Val Acc (%)	Final Val Acc (%)	Final Train Acc (%)	Overfitting Gap (%)	Epochs	Time (min)
Chroma	67.12	65.09	89.75	24.66	33	2.8
Hybrid	57.21	56.39	87.61	31.23	42	7.7
CQT60	53.47	52.21	87.49	35.28	42	11.5
CQT84	49.50	48.33	84.33	36.00	33	12.1

Baselines: - Random guessing: 8.3% (root-only), 2.5% (root+quality) - Best models: 79.61% (root-only), 67.12% (root+quality) - **Improvement:** 9.6x over random (root), 26.8x over random (quality)

7.2 Feature Type Analysis

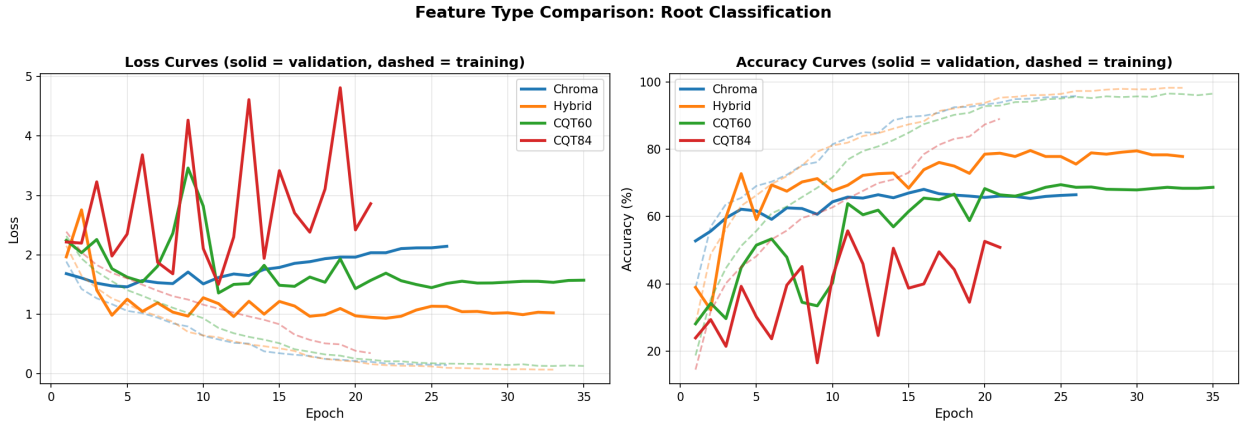


Figure 5: Root Classification Comparison

7.2.1 Hybrid Features: Winner for Root Classification

The Hybrid approach achieved the best root-only performance at 79.61% validation accuracy. This validates the hypothesis that **combining bass-specific information (Low-CQT) with harmonic summary (Chroma)** provides complementary information optimal for root detection.

Why it works: - Low-CQT preserves bass note distinctiveness (where roots live) - Chroma adds harmonic context - 36 bins strike a balance: informative but not overly complex - Lowest overfitting gap (20.37%) among high-performing models

7.2.2 Chroma: Surprisingly Strong for Quality

Despite being the simplest feature, Chroma achieved the best root+quality performance at 67.12%. This surprising result reveals an important insight about chord quality recognition.

Why it works: - Chord quality is defined by **intervals** (relationships between notes), not absolute frequencies - Chroma's octave-invariant representation naturally captures interval relationships - Example: Major third (C-E) appears the same in any octave - Simplicity is an advantage: less overfitting risk with 40 classes

7.2.3 CQT84: The Overfitting Problem

CQT84 performed worst on both tasks (55.78% root, 49.50% quality) despite having the most frequency information.

Why it failed: - 84 bins with only ~1,880 training samples → severe overfitting - Overfitting gap of 38.13% (highest among all models) - High-frequency bins (above C6) add noise without musical value - Complexity without benefit

Key Learning: More features does not equal better performance, especially with limited data.

7.2.4 CQT60: The Goldilocks Approach

CQT60 performed moderately well (69.48% root), validating that removing high frequencies helps.

Observation: - Better than CQT84 and Chroma for root-only - Slower than Hybrid without accuracy gain - Demonstrates that feature subset selection matters

7.3 Training Dynamics Analysis

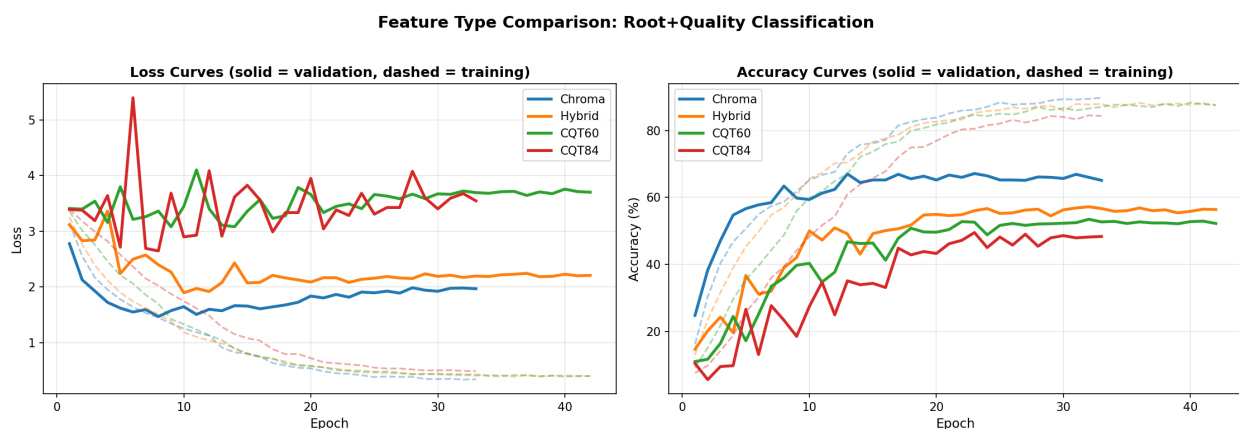


Figure 6: Root+Quality Classification Comparison

Loss Curves Observations:

1. **Hybrid (root-only):** Smooth convergence, stable validation loss
2. **CQT84 (root-only):** Wild fluctuations, unstable training suggesting overfitting
3. **Chroma (root+quality):** Steady decrease, good generalization
4. **All (root+quality):** Plateauing around epoch 20-30 indicates task difficulty ceiling

Accuracy Curves Observations:

1. **Training accuracy consistently higher than validation** - Indicates some degree of overfitting across all models
2. **Validation accuracy plateaus** - Early stopping prevented wasteful training
3. **CQT84 shows erratic validation accuracy** - Confirms overfitting issues

7.4 Task Difficulty Comparison

Root-Only vs Root+Quality: - Root-only: Best model at 79.61% (Hybrid) - Root+quality: Best model at 67.12% (Chroma) - **Insight:** Quality is harder to classify than roots (as expected)

Why quality is harder: 1. More classes (40 vs 12) → lower theoretical maximum 2. Quality depends on subtle interval relationships 3. Acoustic prominence favors roots (bass emphasis) 4. Dataset imbalance (some qualities rarer than others)

7.5 Computational Efficiency

Speed-Accuracy Tradeoff: - Fastest: Chroma (2.3 min) - 68.10% accuracy - Best accuracy: Hybrid (6.0 min) - 79.61% accuracy - Slowest: CQT60/84 (9-12 min) - worse accuracy than Hybrid

Conclusion: Hybrid offers the best speed/accuracy balance for root classification.

8. Discussion and Future Work

8.1 What Worked Well

1. Domain-Specific Feature Engineering The custom Hybrid feature outperformed both simpler (Chroma) and more complex (CQT84) approaches. This demonstrates that musical knowledge directly informed better features: - Bass region for root detection - Chroma for harmonic structure - Balanced complexity for dataset size

2. Data Augmentation Strategy Pitch-shift augmentation effectively quadrupled dataset size while maintaining musical validity. Critically, grouped splitting prevented data leakage.

3. CNN Architecture Design The 5×1 vertical kernel in the first convolutional layer successfully exploited musical structure. Adaptive pooling allowed multi-feature experimentation without architecture redesign.

4. Systematic Experimentation Testing four feature types with identical training procedures enabled fair comparison and clear insights about what works.

8.2 Limitations and Challenges

1. Dataset Size

Problem: ~2,200 samples (post-augmentation) is small for deep learning.

Evidence: - High overfitting gaps (20-38%) - Training accuracy » validation accuracy - Plateauing validation curves

Impact: Models likely memorizing training patterns rather than learning generalizable features.

What's Needed: Ideally 10,000+ samples from diverse songs, genres, recording conditions.

2. Jazz Music Complexity

Problem: Jazz harmony is intricate and context-dependent.

Specific Challenges: - **Chord changes within bars:** Splitting bars with 2 chords loses temporal context - **Voice leading:** Smooth note transitions between chords provide important cues - **Harmonic ambiguity:** Some chords are inherently ambiguous (e.g., rootless voicings) - **Extended harmonies:** 9ths, 11ths, 13ths were normalized, losing nuance

Example: In a ii-V-I progression ($Dm7 \rightarrow G7 \rightarrow Cmaj7$), the $G7$ strongly implies resolution to C . Isolated bars miss this context.

What's Needed: Sequence models that consider chord progressions, not just individual chords.

3. Spectrograms as “Pseudo-Images”

Problem: Treating audio spectrograms as images is indirect.

Why it's suboptimal: - Music has inherent temporal structure (sequences of chords) - CNNs designed for spatial patterns, not temporal dependencies - Time dimension treated as just another spatial dimension

Better Approaches: - **Recurrent networks (RNNs, LSTMs):** Model temporal dependencies - **Transformers:** Attention mechanisms for long-range relationships - **Hybrid CNN-RNN:** CNN for local patterns + RNN for sequences - **Direct audio models:** Skip spectrograms entirely (e.g., WaveNet, SampleRNN)

4. Recognition vs Generation

Philosophical Issue: Recognizing individual chords doesn't solve the original problem (chord chart generation).

Why: - Chord charts need: - Song structure (verse, chorus, bridge) - Timing (where chord changes occur) - Musical form (how many bars, repeats, codas) - This project only recognizes pre-segmented bars

What's Needed: Full end-to-end system that: 1. Analyzes song structure 2. Detects chord change timings 3. Recognizes chords at each position 4. Formats as playable chart

8.3 Future Improvements

1. Larger, More Diverse Dataset

Actionable Steps: - Expand to 50-100 jazz standards - Include multiple recordings per song (different artists, tempos) - Add other genres (pop, funk, R&B have simpler harmony) - Incorporate public datasets (e.g., Isophonics, Billboard chord annotations)

Expected Benefit: Better generalization, lower overfitting, higher accuracy.

2. Sequence-Based Models

Proposed Approach: - Replace CNN with or augment with temporal models - Bidirectional LSTM to process chord progressions - Predict current chord conditioned on previous/next chords - Model entire song structure, not isolated bars

Expected Benefit: Leverage harmonic context, recognize patterns like ii-V-I.

3. Multi-Task Learning

Proposed Approach: - Simultaneously predict: - Root note - Chord quality - Next chord (prediction task) - Key signature - Shared representation learning across tasks

Rationale: Related tasks provide regularization and richer representations.

4. Recognize Larger Musical Structures

Key Insight: Jazz musicians think in **progressions**, not isolated chords.

Common Patterns: - **ii-V-I:** Dm7 $\rightarrow$ G7 $\rightarrow$ Cmaj7 (resolution pattern) - **I-vi-ii-V:** Cmaj7 $\rightarrow$ Am7 $\rightarrow$ Dm7 $\rightarrow$ G7 (turnaround) - **Bird Blues:** Specific 12-bar structure with characteristic changes

Proposed Approach: - Train model to recognize these macro-patterns - Multi-bar context windows - Hierarchical model: local (chord) + global (progression)

Expected Benefit: Higher accuracy by exploiting musical grammar.

5. Address Class Imbalance

Problem: Not all root notes appear equally (C, F, G, Bb are common; Db, Gb are rare).

Proposed Solutions: - Weighted loss function (higher weight for rare classes) - Oversampling rare classes - More targeted data collection for underrepresented roots

Expected Benefit: Better performance on infrequent chords.

8.4 Broader Implications

For Music Information Retrieval: This project demonstrates that domain knowledge is critical for audio ML: - Feature engineering based on music theory outperforms generic approaches - CNN architectures can be customized for musical structure - Small datasets can yield useful results with careful augmentation

For Deep Learning: Lessons applicable beyond music: - More complex features does not equal better performance (CQT84 lesson) - Domain-specific architecture design matters (5×1 kernel) - Data leakage prevention is critical (grouped splitting) - Interpretable features (spectrograms) aid debugging

For Musicians: While not a complete solution, this work demonstrates feasibility of automated chord recognition, bringing automatic chord chart generation closer to reality.

9. Conclusion

This project successfully implemented a CNN-based system for chord root note recognition from audio, achieving 79.61% validation accuracy on a 12-class task—nearly 10x better than random guessing. Through systematic experimentation with four feature extraction methods, the custom Hybrid approach (combining Low-CQT and Chroma) emerged as the clear winner for root classification, while simpler Chromagram features surprisingly excelled at root+quality classification.

Key technical achievements include:

1. **Domain-Specific Feature Engineering:** Designing the Hybrid feature based on musical principles (bass prominence, harmonic structure)
2. **Musically-Informed Architecture:** Vertical 5×1 kernels to capture harmonic intervals
3. **Rigorous Experimental Methodology:** Grouped train/val splitting, systematic comparison, proper augmentation
4. **Practical Engineering:** Refactoring from complex cluster workflow to streamlined Kaggle deployment

The project also revealed important limitations: small dataset size led to overfitting, isolated bar segments missed harmonic context, and treating spectrograms as images may be suboptimal for inherently temporal music. Future work should explore sequence models (LSTMs, Transformers), larger datasets, and recognition of musical progressions rather than isolated chords.

Ultimately, this project demonstrates that machine learning can successfully tackle musical tasks when guided by domain expertise. While automatic chord chart generation remains an ambitious goal, this work establishes a solid foundation and provides a roadmap for future improvements.

As a working musician and aspiring ML engineer, completing this end-to-end project—from data collection through feature engineering to model deployment—has been both musically and technically rewarding. The intersection of music theory and deep learning offers rich opportunities for future exploration.

10. References

Libraries and Tools

- **PyTorch:** Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS*.
- **librosa:** McFee, B., et al. (2015). librosa: Audio and Music Signal Analysis in Python. *SciPy*.
- **NumPy:** Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*.
- **scikit-learn:** Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*.
- **Ableton Live:** Audio production software for data collection
- **Kaggle:** Cloud platform for GPU training
- **iReal Pro:** Mobile app for chord charts (data source)

Dataset

- Jazz standards: Public domain compositions
- Audio recordings: YouTube + iReal Pro Generated Recordings
- Chord progressions: iReal Pro Charts (community database)

Code Repository Structure

Environment/

Chord_Recognition/	# Refactored Kaggle version
data_preprocessing.ipynb	# All preprocessing
model_training.ipynb	# Training on Kaggle
data_input/00_raw/	# Input audio

data_output/*.csv	# CSV features
feature_visualizations/	# Example spectrograms
Chord_Recognition_Old/	# Original cluster version
pipelines/*.py	# 7 pipeline scripts
src/	# Python library
data/	# 5-stage pipeline
results90/	# Experimental results
comprehensive_comparison_report.csv	
feature_comparison_root_classification.png	
feature_comparison_root_quality_classification.png	
CLAUDE.md	# Project documentation
REFACTORING_SUMMARY.md	# Refactoring details
REPORT_NOTES.md	# Report preparation notes
data_pipeline_diagram.png	# Generated diagram
cnn_architecture_diagram.png	# Generated diagram
FINAL_REPORT.md	# This document

Hyperparameter Configuration

Training Hyperparameters: - Loss: CrossEntropyLoss - Optimizer: Adam - Initial Learning Rate: 0.001 - LR Scheduler: ReduceLROnPlateau (factor=0.5, patience=5) - Batch Size: 32 - Max Epochs: 100 (with early stopping) - Early Stopping Patience: 10 epochs - Dropout: 0.5

Data Configuration: - Sample Rate: 22,050 Hz - Hop Length: 512 samples - Augmentation Shifts: [-2, 0, +2, +4] semitones - Train/Val Split: 80/20 (grouped)

Feature Parameters: - Chroma: 12 bins, 87 frames - Hybrid: 24 Low-CQT + 12 Chroma = 36 bins, 87 frames - CQT60: 60 bins, 5 octaves, 87 frames - CQT84: 84 bins, 7 octaves, 87 frames
